

Home Lab Project Week 7, Day 4

Task:

Review available system/network logs and correlate them with the Wireshark capture and exploitation activity. Document the evidence and security implications of the successful intrusion..

This task assumes a successful intrusion to document — and that's the thing to pause on first, because the exploitation I ran earlier didn't succeed. Neither the **vsftpd** backdoor was correctly blocked (my target runs the patched 3.0.5), and no session was created. So right now there's no real intrusion to correlate logs against, Nor **PostgreSQL**. Consequently, I can't document evidence of an event that didn't happen, and So I have two honest routes, and the DVWA work I am setting up gives me the cleaner one.

Route 1 — produce a real successful intrusion with DVWA, then document that. Once DVWA is running, its Command Injection module (at Low security) lets you actually execute commands on the Ubuntu server through the web app. That *is* a genuine intrusion, and it leaves real, correlatable traces in three places at once — which is exactly what this task wants. This is the route I prefer because it makes the task true.

Route 2 — documenting the vsftpd attempt as a blocked intrusion. The task "exploitation attempt and defensive analysis," and correlate the logs showing the attempt and why it failed.

Assuming Route 1, here's the methodology — the transferable skill is correlating the same event across layers.

The three evidence sources you're tying together:

The **Wireshark capture** is the network layer — the raw HTTP request carrying your injected command crossing the wire from Kali to Ubuntu. Capture on the Ubuntu-facing interface while you run the attack.

The **web server logs** on Ubuntu are the application layer. Apache records every request:

```
sudo cat /var/log/apache2/access.log | grep command
```

You'll see the GET/POST request to the command-injection page, with your attacker IP, the timestamp, and often the injected payload visible in the URL.

The **system logs** are the host layer — evidence the injected command actually executed. Depending on what you inject:

```
sudo journalctl --since "10 min ago"  
sudo cat /var/log/auth.log
```

```
msf > search vsftp

Matching Modules

#  Name                                     Disclosure Date  Rank    Check  Description
-  -                                     -              -      -      -
0  auxiliary/dos/ftp/vsftpd_232             2011-02-03      normal Yes    VSFTPD 2.3.2 Denial of Service
1  exploit/unix/ftp/vsftpd_234_backdoor      2011-07-03      excellent Yes    VSFTPD 2.3.4 Backdoor Command Execution

Interact with a module by name or index. For example info 1, use 1 or use exploit/unix/ftp/vsftpd_234_backdoor
```

No.	Time	Source	Destination	Protocol	Length	Info
17	22.170693294	192.168.78.3	192.168.78.5	TCP	74	5432 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2500745633 TSecr=0 WS=1024
18	72.171565732	192.168.78.5	192.168.78.3	TCP	60	5432 → 33107 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
23	84.592739022	192.168.78.3	192.168.78.5	TCP	74	33447 → 5432 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1868771386 TSecr=0 WS=1024
24	84.594285775	192.168.78.5	192.168.78.3	TCP	60	5432 → 33447 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
27	181.218548871	192.168.78.3	192.168.78.5	TCP	74	38779 → 5432 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=965128253 TSecr=0 WS=1024
28	181.219322463	192.168.78.5	192.168.78.3	TCP	60	5432 → 38779 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

```
msf exploit(unix/ftp/vsftpd_234_backdoor) > set RHOSTS 192.168.78.5
RHOSTS => 192.168.78.5
msf exploit(unix/ftp/vsftpd_234_backdoor) > █
```

Time	Source	Evidence	What it proves
	Wireshark	HTTP POST with ;id payload	The attack crossed the network to the target
	Apache access.log	Same request logged, same attacker IP	The server received and processed it
	command output	uid=33(www-data) returned	The command executed as the web server user

That matching timestamp and IP across all three layers is the proof. It shows the same single event seen from three vantage points.

Security implications to write up (the "so what"): the attacker ran commands as `www-data`, meaning any data that user can read is exposed; cleartext HTTP meant the whole attack was visible in the capture; and a defender watching these same logs could have detected it — which is the lesson. Then give remediation: input validation/sanitization, running the web app with least privilege, and HTTPS.